

Spring AI: Modül 05

RAG Bölüm 1: Vektör Gömme (Embeddings), Vektör Veritabanları ve Doküman ETL

Kurumsal Java Yapay Zeka Mühendisliği

1. RAG Nedir? (Retrieval-Augmented Generation)

- Büyük Dil Modellerinin (LLM) bilgi kesim tarihleri vardır ve kurum içi özel verileri bilmezler.
- **RAG** bu sorunu çözer:
 1. Vektör veritabanından en alakalı kurum içi bilgi parçalarını çeker.
 2. Bu parçaları prompt bağlamına (context) ekler.
 3. Halüsinasyonsuz ve kanıtlanabilir yanıtlar üretir.

2. Yoğun Vektör Gömme (Dense Vector Embeddings)

- Metinler sabit boyutlu sayısal vektör dizilerine dönüştürülür (örn. 384, 768, 1536 float).
- Anlamsal olarak birbirine yakın kavramlar vektör uzayında birbirine komşudur.
- Benzerlik ölçümünde genellikle **Kosinüs Benzerliği (Cosine Similarity)** kullanılır.

3. Spring AI Doküman ETL Mimarisi

```
[Ham Doküman] -> [DocumentReader]  
               -> [TokenTextSplitter]  
               -> [EmbeddingModel]  
               -> [VectorStore (pgvector / Simple)]
```

- **Document** : Metin içeriğini ve anahtar-değer üst verilerini (metadata) taşır.
- **TokenTextSplitter** : Metinleri token sınırlarına göre mantıklı parçalara böler.

4. İçe Aktarma (Ingestion) Kod Örneği

```
Document rawDoc = new Document(policyText, Map.of(
    "department", "Security",
    "version", "2026.1"
));

List<Document> chunks = tokenTextSplitter.apply(List.of(rawDoc));
vectorStore.accept(chunks);
```

5. Eşik Değerli Benzerlik Araması

```
SearchRequest request = SearchRequest.builder()  
    .query("VPN'e nasıl bağlanılır?")  
    .topK(3)  
    .similarityThreshold(0.70)  
    .filterExpression("department == 'Security'")  
    .build();  
  
List<Document> results = vectorStore.similaritySearch(request);
```

6. Özet ve Çıkarımlar

1. Büyük dokümanları doğrudan bağlama atmayın; akıllıca parçalayın (chunking).
2. Üst veriler (metadata) ile ön filtreleme yaparak arama kalitesini artırın.
3. Eşik değeri (threshold) kullanarak alakasız sonuçların yanıtı girmesini engelleyin.